

EMAP

An Exploratory Artifact Report on Resource-Constrained Evolution of Multi-Agent Programming Architectures

Noah Ingwers | December 2025 artifacts | Research note revised August 20, 2026

Evidence scope: 12 checked-in runs, four token budgets, three seeds per budget, and 41 sampled final tasks per run. This is not a full HumanEval evaluation and does not support causal or deployment claims.

Abstract

EMAP explores whether token budgets used as evolutionary pressure are associated with different multi-agent programming architectures. This note reports only what can be supported by the twelve JSON artifacts checked into the repository. The artifacts are exploratory and non-peer-reviewed. They support descriptive comparisons, not causal conclusions, statistical significance, universal architecture recommendations, or production guidance.

Artifact scope

The repository contains twelve completed runs in `experiments/results/focused/`: TIGHT (2,000 tokens), MEDIUM (5,000), LOOSE (10,000), and UNCONSTRAINED (50,000), each with seeds 42, 43, and 44. The checked-in summary records 12 generations, a population of 10, a 12-percent task sample during evolution, and a 25-percent final sample. Each per-run artifact reports `final_total`: 41.

Across the twelve artifacts, the recorded totals are **3,093,390 tokens** and **27,360 API calls**. Recorded runtimes range from approximately **1.7 to 5.2 hours** per run. Input and output tokens are not separated, so historical dollar cost cannot be reconstructed from the artifacts.

Method represented by the artifacts

The runner was intended to use the 164-task HumanEval corpus, sampling tasks during evolution and for final evaluation. A 41-task final total is consistent with the configured 25-percent sample, but the artifacts do not preserve task identifiers, per-task outputs, or a dataset hash. The recorded scores therefore are not presented as full-HumanEval results, and the evidence cannot establish that different seeds used identical task subsets.

The seed controls NumPy-based architecture evolution. Task selection uses Python's `random` module without a recorded seed, and remote model calls are nondeterministic. The model name is absent from the JSON artifacts; GPT-4o-mini attribution comes from the checked-in runner configuration rather than from a `model` field in each record.

Recorded results

Regime	Budget	Mean final score	Mean agents	Mean edges
TIGHT	2,000	98.4% +/- 2.3%	3.2	2.6
MEDIUM	5,000	100.0% +/- 0.0%	3.1	2.4
LOOSE	10,000	98.4% +/- 2.3%	3.5	2.8
UNCONSTRAINED	50,000	99.2% +/- 1.1%	2.9	2.1

Table 1. Means and population standard deviations across three recorded runs per regime. Agent and edge counts are final-generation population averages, not the size of one selected architecture.

All three MEDIUM artifacts report 41 of 41 sampled final tasks passed. The three MEDIUM runs also record 414,390 total tokens, the lowest aggregate among the four regimes. In this small, uncontrolled sample, those values make MEDIUM the strongest observed regime. They do not establish 5,000 tokens as an optimum or justify a general Goldilocks-zone claim.

The final populations averaged roughly three agents across regimes. Selected genomes in the records range from two to four agents and include linear and cyclic structures with different roles, aggregation modes, and message formats. These observations document diversity in the recorded outcomes; they do not show that budget pressure caused that diversity or that three-agent systems are generally optimal.

Reproducibility limits

- There are three runs per regime, with no preregistered analysis or statistical significance test.
- Final task identifiers, per-task outputs, prompt transcripts, dataset hashes, Git commits, dependency versions, and model versions are not preserved.
- Task sampling and remote model responses are not fully seeded, so exact reruns are not guaranteed.
- The project specifies dependency lower bounds and has no lockfile for the December 2025 environment.
- The study does not evaluate a full benchmark, cross-benchmark transfer, other models, or deployment performance.

Execution-harness safety

The benchmark harness evaluates generated Python with `exec` inside a disposable subprocess. It applies static denylist checks, restricted builtins and imports, a parent-process timeout, and best-effort Unix limits for CPU, address space, file size, and file descriptors. These controls reduce accidental risk but are **not a secure sandbox**. Python object-model escapes and platform differences remain possible.

Run the evaluator only on an unprivileged, isolated host without secrets or sensitive network/filesystem access. The implementation fails closed if subprocess isolation cannot be established; it does not fall back to executing generated code in the caller process.

Conclusion

The twelve artifacts are a compact exploratory record. They show high sampled-task scores and several architecture shapes across the recorded budget/seed matrix. The evidence is not sufficient for causal claims, full-HumanEval performance claims, universal architecture recommendations, or historical cost estimates. A stronger follow-up would preregister task splits, record per-task evidence and environment metadata, fully seed the pipeline, and repeat the comparison across models and benchmarks.

Availability and license

The aggregate summary and per-run JSON records are checked into the repository. The software and this note are distributed under the repository's MIT License.